

COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs

Anonymous Author(s)

Abstract

In this paper, we explore time-interval-based community search over temporal graphs, which aims to identify a connected cohesive subgraph containing the query vertices within a specific time interval. Existing work suffers from two main limitations: (1) *information loss*, as it ignores the interaction frequency between vertices, a crucial factor for identifying high-quality communities. (2) *efficiency bottleneck*, as it either incurs high search latency from expensive online computation or faces a combinatorial explosion in indexing time and storage when precomputing coreness values across all possible time intervals. To address these issues, we propose COCO, a cohesiveness-aware learning framework for time-interval-based community search over temporal graphs. COCO works in three phases: pre-training, fine-tuning, and searching. It contains two novel components: (1) a *hierarchical model index*, which efficiently computes coreness across different time intervals via a hierarchical model tree, and (2) an *adaptive temporal GNN*, which effectively captures both graph structural information and interaction frequencies while supporting efficient fine-tuning. Experiments on eight real-world datasets show that COCO outperforms state-of-the-art methods in community quality and is at least 10× faster than the baselines, demonstrating its effectiveness and efficiency.

ACM Reference Format:

Anonymous Author(s). 2026. COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (SIGIR '26)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Community search is an important tool in graph analysis, aiming to identify cohesive subgraphs that contain given query vertices [8, 28]. Prior research has extensively investigated community search on various graph types, including directed graphs [3, 9, 24], bipartite graphs [23, 39, 40], attributed graphs [25, 42], and multilayer graphs [2, 26, 35].

In real-world applications, interactions between entities are naturally dynamic and change over time. For instance, in social networks, user activities, such as comments, likes, and shares, are timestamped, reflecting the temporal nature of relationships. This temporal information offers a unique perspective for understanding user behavior patterns and community structures. As a result,

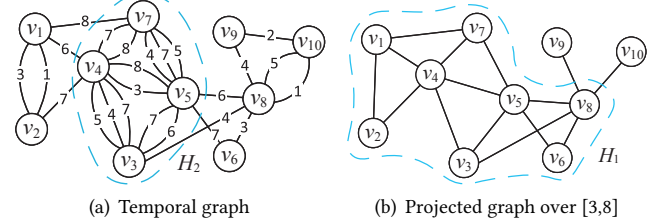


Figure 1: A motivating example

community search over temporal graphs has garnered significant research interest [19–22, 44, 47], offering a more nuanced understanding of network dynamics. Among them, [44] identifies communities within specific time intervals on temporal graphs. Specifically, given a temporal graph, a query vertex, a specified time interval, and an integer k , the method in [44] returns a connected k -core that includes the query vertex in a projected graph. This projected graph is constructed by retaining only the edges within the specified time interval, and multiple edges between the same pair of vertices are merged into one. For example, Figure 1(a) shows a temporal graph where each edge is timestamped to indicate when the corresponding interaction occurred. Given the query vertex v_4 , the time interval $[3, 8]$, and $k = 2$, the method in [44] identifies community H_1 in the corresponding projected graph, as shown in Figure 1(b).

Limitations. Existing time-interval-based community search over temporal graphs suffers from two major limitations.

Information Loss. The method in [44] identifies communities directly in the projected graph for a given time interval. However, this projected graph fails to capture the interaction frequency between vertices. Interaction frequency (the number of distinct temporal edges between two vertices) is a direct measure of connection strength; groups with higher interaction frequencies signify tighter relationships and thus form more cohesive communities. Ignoring this frequency leads to overlooking densely connected subgraphs, which are essential for identifying high-quality communities. For example, in Figure 1(a), H_2 is a better community for v_4 within the time interval $[3, 8]$, as each vertex in H_2 interacts with its neighbors an average of 2.4 times, indicating a more cohesive group.

Efficiency Bottleneck. The method in [44] faces a severe efficiency dilemma. Performing online core decomposition for each query-specific interval is computationally expensive, leading to unacceptable search latency on large-scale graphs. For instance, performing online core decomposition on a graph with millions of vertices may take around 500 seconds (see Section 6.2 for detailed experimental results). Alternatively, while indexing can accelerate queries, it requires precomputing and storing results for all possible k values and time intervals. Due to the quadratic growth of time intervals, such an approach suffers from prohibitive preprocessing time and memory overhead, rendering it impossible to construct the index

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR '26, Melbourne, Australia

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

for large-scale temporal graphs. Specifically, the index for a graph with two million vertices can exceed 1 GB, and for even larger graphs, it cannot be constructed within 24 hours.

To address these limitations, we revisit and formalize the *Time-Interval-based Community Search (TICS)* problem. Specifically, given a temporal graph $\mathcal{G}$, a time interval $[t_s, t_e]$, and a set of query vertices V_Q , the goal of TICS is to identify a community containing V_Q within $[t_s, t_e]$.

Challenges. The TICS problem presents two main challenges.

Challenge I. How to identify high-quality communities? Each TICS query can specify a unique time interval, leading to distinct graph topologies and community structures. This inherent variability makes identifying high-quality communities challenging. An effective TICS approach must not only accurately capture temporal interactions to reflect dynamic relationships but also flexibly adapt to the diverse graph structures that arise from different queries.

Challenge II. How to address TICS efficiently? The variability in graph structures across different queries also introduces significant efficiency challenges. Processing each TICS query from scratch is computationally expensive, especially for large-scale temporal graphs. While preprocessing or indexing techniques could help, they would need to account for the vast number of potential graph structures spanning all possible time intervals, which adds another layer of computational complexity.

Our Solutions. To overcome these challenges, we propose **COCO**—a novel learning framework for **C**ohesiveness-aware **C**ommunity search over temporal graphs. COCO employs a three-stage design (pre-training, fine-tuning, and searching) that seamlessly integrates the classical k -core model with graph neural networks (GNNs). This approach offers three key advantages: (1) it efficiently identifies communities with both strong structural cohesiveness and frequent interactions; (2) it adapts flexibly to diverse time intervals without being constrained by fixed structural requirements; and (3) it effectively balances the trade-off between community quality and query processing speed.

Specifically, COCO incorporates two novel components: the *Hierarchical Model Index* (HM-Index) and the *Adaptive Temporal GNN* (AT-GNN). (1) The HM-Index is designed for efficient vertex coreness computation across arbitrary time intervals. It replaces traditional, computationally expensive core decomposition algorithms [28] with an innovative learning-based approach. This is critical because reapplying decomposition algorithms from scratch for every query-specific graph is prohibitive. The HM-Index organizes deep learning models into a hierarchical tree, where each node contains a model trained for a specific time interval. To predict coreness for a new query interval, it traverses a path from the root, iteratively applying the models along the path. This hierarchical design enables highly accurate and scalable coreness prediction across diverse time intervals. (2) The AT-GNN is a graph neural network specifically designed to capture both structural information and vertex interaction frequencies. By integrating the coreness values from the HM-Index with interaction frequencies, it significantly enhances the model’s ability to identify cohesive communities. Furthermore, the AT-GNN incorporates an adapter module, which allows it to rapidly fine-tune and adapt to the unique

graph structures of different queries without requiring a full re-training process.

The COCO framework operates through a three-phase pipeline. First, in the pre-training phase, COCO (1) trains coreness prediction models on different time intervals and then organizes these models into an HM-Index; and (2) trains AT-GNN in an unsupervised manner, enabling it to learn common community patterns across time intervals. We design multiple attention mechanisms and two specialized loss functions to ensure that AT-GNN effectively captures interaction frequencies between vertices during the training process. Then, in the fine-tuning phase, COCO predicts the coreness for query vertices and their neighbors via traversing the HM-Index and adjusts AT-GNN’s adapter module parameters using these corenesses and the query context. In the searching phase, COCO uses vertex embeddings from AT-GNN to iteratively select vertices most similar to the query vertices, forming the final community.

Contributions. Overall, our main contributions are as follows.

- We propose COCO, a novel cohesiveness-aware learning framework for time-interval-based community search (TICS). Using a three-stage pipeline, it identifies high-quality communities with strong structural cohesiveness and frequent interactions.
- We design the *Hierarchical Model Index* (HM-Index), a learning-based component for efficiently computing vertex coreness across any time interval. It arranges lightweight models in a hierarchical tree, eliminating the high preprocessing time and storage costs of traditional indexing methods.
- We develop the *Adaptive Temporal GNN* (AT-GNN) to capture graph structure and temporal interaction frequencies. It includes a query-aware adapter module, allowing quick adaptation to different query structures without retraining.
- We conduct extensive experimental studies on eight real-world graphs. The results validate the superior effectiveness and efficiency of our proposed COCO framework.

2 Related Work

We review the related work from two aspects, including classic community search over temporal graphs, and learning-based methods for community search.

Classic community search over temporal graphs. Based on temporal graph characteristics, various community search methods have been studied. For instance, time-interval-based approaches [19, 44, 47] aim to find communities in the projected graph over a given time interval. [20, 21] assume graphs are snapshots and require communities to exist in every snapshot within the interval. Others, like [22], focus on query-centered search by combining temporal proximity and structural cohesion. Several cohesive subgraph models have also been proposed, including span core [10, 11], periodic clique [29], periodic core [31], temporal k -core [43, 45, 48], and bursting core [30]. However, these do not support query-based search. In this work, we focus on time-interval-based community search and propose COCO, a new framework that overcomes limitations of prior methods discussed in Introduction.

Learning-based methods for community search. The existing learning-based community search methods can be mainly divided

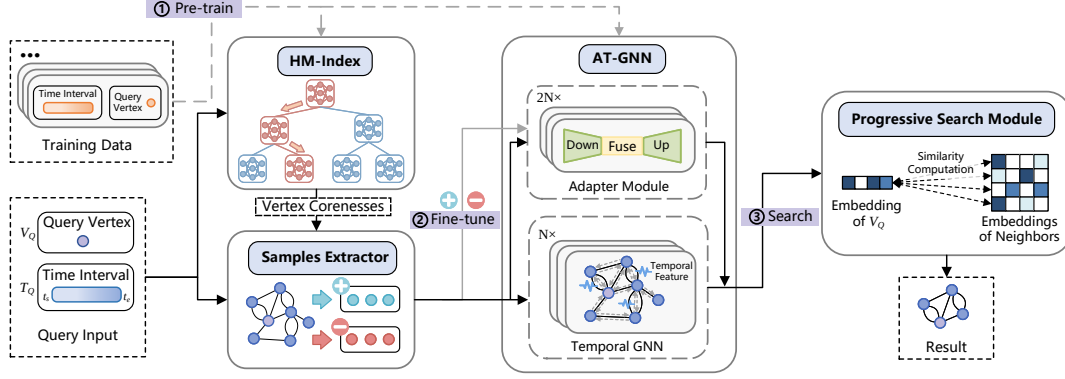


Figure 2: The design of COCO framework

into three categories: supervised methods, semi-supervised methods, and unsupervised methods. The supervised methods [7, 14, 38] depend on ground-truth communities, which are used to train models to learn community structures. The semi-supervised methods [18, 34] rely on only a fraction of labeled vertices instead of complete ground-truth labels. The unsupervised methods [37, 41] rely on contrastive learning or heuristic rules to produce pseudo-labels for pretraining, circumventing the requirement for ground-truth communities. Existing learning-based methods struggle with generalization in the TICS problem. They work well on static graphs but fail on dynamic or unseen graph structures.

Recently, Hashem et al. [12] proposed CS-TGN for community search based on the temporal GNN. Zhang et al. [46] proposed UTCS, an unsupervised method for temporal community search. Jiang et al. [15] devised PK-GCN to identify the persistent k -core in the temporal graph based on GNN. Compared to COCO, CS-TGN and UTCS have limited ability to capture temporal information, and ignores the importance of community cohesiveness. PK-GCN still encounters the structure inflexibility problem, since it essentially uses deep learning models to find k -core structures.

3 Preliminary

A temporal graph is represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ denote the sets of vertices and temporal edges, respectively. Each edge $e \in \mathcal{E}$ is a triplet (u, v, t) , representing an interaction between vertices u and v at timestamp t . The entire time interval of $\mathcal{G}$ is $\mathcal{T} = [1, t_{\max}]$. For a given time interval $[t_s, t_e] \subseteq \mathcal{T}$, the temporal graph $\mathcal{G}$ over $[t_s, t_e]$ is denoted by $\mathcal{G}_{[t_s, t_e]} = (\mathcal{V}_{[t_s, t_e]}, \mathcal{E}_{[t_s, t_e]})$, where $\mathcal{V}_{[t_s, t_e]} = \mathcal{V}$ and $\mathcal{E}_{[t_s, t_e]} = \{(u, v, t) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. The projected graph of $\mathcal{G}$ over $[t_s, t_e]$ is denoted by $\mathcal{G}_{pro[t_s, t_e]} = (\mathcal{V}_{pro[t_s, t_e]}, \mathcal{E}_{pro[t_s, t_e]})$. Here, $\mathcal{V}_{pro[t_s, t_e]} = \mathcal{V}$ and $\mathcal{E}_{pro[t_s, t_e]} = \{(u, v) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. We use $N(v)$ to denote the neighbors of vertex v in $\mathcal{G}$. For the neighbors of v over a specific time interval $[t_s, t_e]$, we denote this set as $N_{[t_s, t_e]}(v)$. The degree of v at a particular timestamp $t \in \mathcal{T}$ is denoted by $d_t(v)$. Additionally, the degree of v over the time interval $[t_s, t_e]$ is $d_{[t_s, t_e]}(v) = |N_{[t_s, t_e]}(v)|$. Based on above notations, we formally define our problem.

Problem 1. (Time-Interval-based Community Search (TICS)). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a time interval $T_Q = [t_s, t_e]$, and a set of query vertices $V_Q \subseteq \mathcal{V}$, the time-interval-based community search aims to find a vertex set C , satisfying:

- (1) $V_Q \subseteq C$;
- (2) The subgraph induced by C in $\mathcal{G}_{[t_s, t_e]}$ is connected;

(3) Vertices in C are densely intra-connected in $\mathcal{G}_{[t_s, t_e]}$.

Definition 3.1. (Temporal k -core [44]). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a time interval $[t_s, t_e]$. Let $\mathcal{G}_{pro[t_s, t_e]}$ be the corresponding projected graph. The temporal k -core over $[t_s, t_e]$ is the maximum subgraph $\mathcal{H} \subseteq \mathcal{G}_{pro[t_s, t_e]}$ such that each vertex in $\mathcal{H}$ has at least k neighbors.

Definition 3.2. (Coreness). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and time interval $[t_s, t_e]$. The coreness of a vertex $v \in \mathcal{V}$ over $[t_s, t_e]$, denoted by $c_{[t_s, t_e]}(v)$, is the largest k value such that v belongs to a temporal k -core in $\mathcal{G}_{pro[t_s, t_e]}$.

Example 1. Take Figure 1 as an example. Let the time interval be $[3, 8]$. The vertices v_1 to v_8 form a temporal 2-core. All vertices v_1 through v_8 share a common coreness value of 2, i.e., $c_{[3, 8]}(v_1) = \dots = c_{[3, 8]}(v_8) = 2$.

Graph Neural Networks. GNNs [36] follow a neighborhood aggregation mechanism, which can be expressed as:

$$\mathbf{h}_v^{(l)} = M\left(\mathbf{h}_v^{(l-1)}, \text{AGG}\left\{\mathbf{h}_u^{(l-1)} : u \in N(v)\right\}\right) \quad (1)$$

Here, $\mathbf{h}_v^{(l)}$ is the representation of vertex v in layer l , AGG is the aggregation function that combines messages from neighbors, and M is the message propagation function that updates the representation of the vertex using the aggregated representations and its own representation from the previous layer.

4 COCO Framework

To address the TICS problem, we present COCO, a novel cohesiveness-aware learning framework. The framework's cohesiveness-aware nature stems from its integration of the classic k -core model with learning-based approaches. Compared to existing learning-based community search approaches, COCO is capable of identifying communities that demonstrate strong cohesiveness and can flexibly adapt to different graph structures across various queries.

Figure 2 shows COCO's three-stage pipeline: pre-training, fine-tuning, and searching. (1) In pre-training, the HM-Index learns to predict vertex coreness over time, while the AT-GNN captures common temporal community patterns. (2) During fine-tuning, the HM-Index predicts coreness for query vertices V_Q and their neighbors in time interval T_Q . A sample extractor then creates positive and negative examples based on these coreness values and the query context, used to fine-tune the AT-GNN's adapter module. (3) In the search phase, AT-GNN generates vertex embeddings. The progressive search module iteratively selects vertices most similar

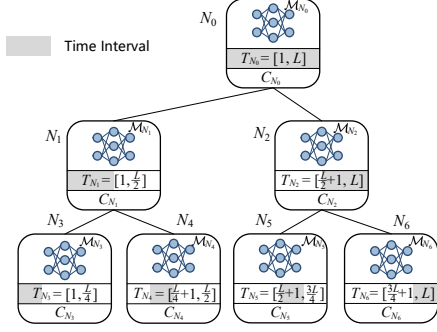


Figure 3: Architecture of HM-Index

to V_Q , adding them to the community until constraints are met. Note that, due to space limitation, some pseudocodes, examples, and proofs are moved to *technical report [1]*.

4.1 Hierarchical Model Index

We propose a learning-based method that predicts coreness instead of computing it directly. Our approach divides the entire time span into smaller intervals and uses lightweight regression models to predict coreness within each. These models are organized hierarchically into a structure called the Hierarchical Model Index (HM-Index), illustrated in Figure 3. Each node N in the HM-Index has three components: (1) T_N : the time interval it covers; (2) $\mathcal{M}_N$: a lightweight model that predicts vertex coreness for any subinterval of T_N ; (3) $C_N = \{c_{T_N}(v) \mid v \in \mathcal{V}\}$: the set of vertex coreness values over T_N , used by the parent node to improve prediction accuracy.

Time Interval Partition Rules. The HM-Index uses recursive partitioning. Specifically, the root covers the full time range $[1, t_{\max}]$. Each node splits its interval into p equal subintervals until the length falls below a threshold x . For example, in Figure 3, $p = 2$ and $x = L/4$. The index maintains two properties: (1) Strict hierarchy: for any child node N' of node N , $T_{N'} \subset T_N$; (2) Disjoint siblings: nodes at the same level cover non-overlapping time intervals.

Model Structure. The regression model $\mathcal{M}_N$ in a node N is a lightweight Multilayer Perceptron (MLP). It consists of four components: (1) $\mathbf{W}_0$, a linear transformation layer for single-valued features; (2) $\mathbf{W}_1$, a timestamp feature processing layer; (3) $\mathbf{W}_2$, a time interval feature processing layer; (4) $\mathbf{W}_f$, a feature fusion layer that merges the three processed features into a unified representation.

Coreness Prediction Algorithm. The coreness prediction algorithm uses a multi-stage approach that traverses the HM-Index from the root, refining predictions at each level. We first describe the input to the model $\mathcal{M}_N$, then present the full algorithm that coordinates this hierarchical prediction process.

Given an HM-Index node N , a vertex v , and a query time interval $[t_s, t_e] \subseteq T_N$, the interval may span multiple child nodes of N . Let $N_c = \{N_0, N_1, \dots, N_{|N_c|-1}\}$ be the ordered sequence of these child nodes, covering a combined interval $[t'_s, t'_e]$. Let N_p be the parent of N . The model $\mathcal{M}_N$ predicts the coreness of v as follows.

$$\hat{c}_{[t_s, t_e]}^N(v) = \mathcal{M}_N(\hat{c}, \mathbf{s}_1, \mathbf{s}_2) = \mathbf{W}_f(\mathbf{W}_0\hat{c} \parallel \mathbf{W}_1\mathbf{s}_1 \parallel \mathbf{W}_2\mathbf{s}_2) \quad (2)$$

The model $\mathcal{M}_N$ takes three inputs to predict the coreness of a vertex v : $\hat{c}$, $\mathbf{s}_1$, and $\mathbf{s}_2$. $\hat{c}$ is the coreness value predicted by the parent node N_p , or 0 if no prediction exists. It helps refine the current prediction by leveraging information from the parent node. The vector

Algorithm 1: Coreness Prediction Algorithm

Input: A temporal graph $\mathcal{G}$, a vertex v , a query time interval $[t_s, t_e]$, HM-Index root node N_{root}

Output: Predicted coreness $\hat{c}$ for v over $[t_s, t_e]$

```

1  $N \leftarrow N_{root}; \hat{c} \leftarrow 0;$ 
2 while True do
3    $\mathbf{s}_1 \leftarrow [c_t(v) \parallel d_t(v)]_{t=t_s, t \notin [t'_s, t'_e]}^{t_e};$ 
4    $\mathbf{s}_2 \leftarrow [C_{T_{N_i}} \parallel d_{T_{N_i}}(v)]_{i=0}^{|N_c|-1};$ 
5    $\hat{c} \leftarrow \mathcal{M}_N(\hat{c}, \mathbf{s}_1, \mathbf{s}_2);$  // Eq. (2)
6   for  $N' \in N.child$  do
7     if  $[t_s, t_e] \subseteq T_{N'}$  then
8        $N \leftarrow N';$  break;
9   if no child node contains  $[t_s, t_e]$  then break;
10 return  $\hat{c};$ 
```

$\mathbf{s}_1$ encodes the coreness and degree of v at timestamps outside a given interval $[t'_s, t'_e]$, capturing external temporal dynamics. The vector $\mathbf{s}_2$ aggregates v 's coreness and degree across all time intervals in the child nodes N_c , providing a summary of its structural properties over time.

Based on the coreness prediction of a single HM-Index node, the full coreness prediction algorithm using the HM-Index is summarized in Algorithm 1. The algorithm starts at the root node N with initial predicted coreness $\hat{c} = 0$ (line 1). For each current node N , it generates feature vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ (lines 3–4) and updates $\hat{c}$ (line 5). It then checks if any child node N' fully covers the query interval $[t_s, t_e]$ (lines 6–8). If so, the algorithm moves to that child; otherwise, it stops. By traversing the index and using temporal and structural features, Algorithm 1 efficiently predicts coreness.

THEOREM 4.1. *The time complexity of Algorithm 1 is $O(\log \frac{t_{\max}}{x})$, and the space complexity of HM-Index is $O(\frac{t_{\max}}{x}(|\mathcal{M}| + |\mathcal{V}|))$.*

4.2 Adaptive Temporal GNN

The Adaptive Temporal Graph Neural Network (AT-GNN) is designed to capture both structural and temporal features of the temporal graph while enabling efficient fine-tuning for various TICS queries. The AT-GNN contains K layers. Figure 4 shows the architecture of one layer in the AT-GNN. The model has two main components: the encoder and the adapter. Each layer includes two adapters, namely the *Pre-aggregation Adapter* and the *Post-aggregation Adapter*. The vertex embedding $h^{(l)}(v)$ in the l -th layer of the AT-GNN is computed as follows.

$$h^{(l)}(v) = \text{ENC}(h^{(l-1)}(v)) + \text{ADP}_{\text{pre}}(h^{(l-1)}(v)) + \text{ADP}_{\text{post}}(\text{ENC}(h^{(l-1)}(v))) \quad (3)$$

Here, $\text{ENC}(\cdot)$ is the encoder, $\text{ADP}_{\text{pre}}(\cdot)$ and $\text{ADP}_{\text{post}}(\cdot)$ are pre-aggregation and post-aggregation adapters, respectively. Next, we introduce them in detail.

Encoder. For a vertex v , the initial feature $\mathbf{h}^{(0)}(v) \in \mathbb{R}^{\dim_v}$ is defined as:

$$\mathbf{h}^{(0)}(v) = f_v([q(v) \parallel \mathbf{s}_c(v)]), \quad (4)$$

where $q(v)$ indicates whether v is a query vertex (1 if $v \in V_Q$, else 0), and $\mathbf{s}_c(v) = [c_t(v)]_{t=t_s}^{t_e}$ is the temporal coreness sequence. The function $f_v(\cdot)$ adjusts dimension via linear interpolation (for short sequences) or average pooling (for long ones). To capture temporal interaction patterns, vertex and edge features are transformed into

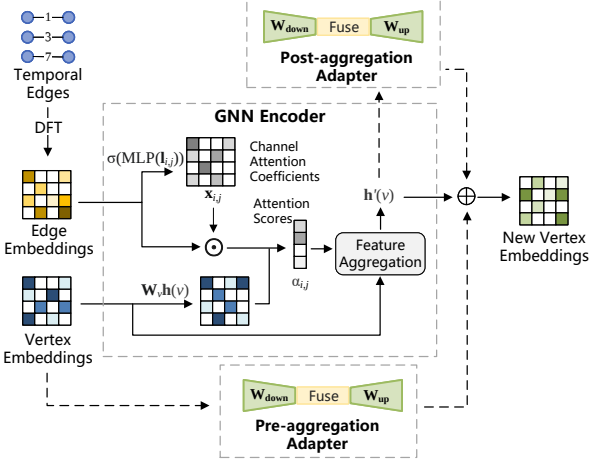


Figure 4: Architecture of AT-GNN

the frequency domain using the Discrete Fourier Transform (DFT). For an edge (v, v') , the initial embedding $\mathbf{I}_{v,v'} \in \mathbb{R}^{\dim_e}$ is

$$\mathbf{I}_{v,v'} = [\text{DFT}(f_e([a_t(v, v')])_{t=t_s}^{t_e})]_{0:\dim_e}. \quad (5)$$

The edge presence sequence $[a_t(v, v')]_{t=t_s}^{t_e}$ indicates whether edge (v, v') exists at each time t . $a_t(v, v') = 1$ if connected, else 0. The function $f_e(\cdot)$ performs dimension transformation, similar to $f_v(\cdot)$. After applying DFT, the first $\dim_e$ elements of the amplitude vector form the edge embedding. To strengthen temporal modeling, a channel attention mechanism refines the embeddings:

$$\mathbf{x}_{v,v'} = \sigma(\text{MLP}(\mathbf{I}_{v,v'})), \quad \mathbf{I}'_{v,v'} = \mathbf{I}_{v,v'} \odot \mathbf{x}_{v,v'}. \quad (6)$$

For vertex aggregation, the encoder calculates attention scores $b_{v,v'}$ by combining vertex and edge features:

$$b_{v,v'} = \text{LeakyReLU}(\mathbf{a}^T \cdot [\mathbf{W}_v \mathbf{h}^{(l-1)}(v) \parallel \mathbf{W}_v \mathbf{h}^{(l-1)}(v') \parallel \mathbf{I}'_{v,v'}]). \quad (7)$$

Here, $\mathbf{a}$ is a learnable vector and $\mathbf{W}_v$ projects vertex features. Dynamic edge features $\mathbf{I}'_{v,v'}$ capture temporal interactions. Then, attention scores are computed and normalized, then used to aggregate neighboring features to update $\mathbf{h}^{(l)}(v)$, as in Eq. (8).

$$\alpha_{v,v'} = \text{softmax}_{v'}(b_{v,v'}) = \frac{\exp(b_{v,v'})}{\sum_{v'' \in \mathcal{N}(v) \cup \{v\}} \exp(b_{v,v''})}, \quad (8)$$

$$\text{ENC}(\mathbf{h}^{(l-1)}(v)) = \sum_{v' \in \mathcal{N}(v) \cup \{v\}} \alpha_{v,v'} \cdot \mathbf{h}^{(l-1)}(v')$$

Adapter. COCO addresses TICS queries with varying graph structures. Fully retraining or fine-tuning the encoder would be too costly. Inspired by parameter-efficient tuning methods in NLP [6, 13], we design a query context-aware adapter. This lightweight, trainable module is inserted into each encoder layer—before and after the aggregation step. The dual-adapter architecture enhances generalization by enabling controlled feature adaptation without altering the original graph representations. It supports flexible query handling while preserving model stability.

As illustrated in Figure 4, each adapter includes three parts: dimension reduction, feature fusion, and dimension expansion. In the dimension reduction stage, two inputs are processed. First, vertex embeddings $\mathbf{h}_{\text{in}}(v)$ are linearly projected to a lower dimension using

$\mathbf{W}_{\text{down}_1}$. Second, query context features are extracted: average coreness of query vertices and their neighbors ($\bar{c}(V_Q)$ and $\bar{c}(N(V_Q))$), and normalized time features $\frac{t_s}{t_{\max}}$ and $\frac{|T_Q|}{|T_{\max}|}$. These features are combined into a vector $\mathbf{s}_Q$, then projected via $\mathbf{W}_{\text{down}_2}$. In the feature fusion module, a self-attention mechanism combines $\mathbf{h}_{\text{ADP}}(v)$ and $\mathbf{s}_{\text{ADP}}$. After concatenation, three learnable matrices $\mathbf{W}_q$, $\mathbf{W}_k$, and $\mathbf{W}_v$ project the input into query, key, and value vectors, as shown in Eq. (9).

$$\tilde{\mathbf{h}}(v) = [\mathbf{W}_{\text{down}_1} \mathbf{h}_{\text{in}}(v) \parallel \mathbf{W}_{\text{down}_2} \mathbf{s}_Q], \quad (9)$$

$$\mathbf{q} = \mathbf{W}_q \tilde{\mathbf{h}}(v), \quad \mathbf{k} = \mathbf{W}_k \tilde{\mathbf{h}}(v), \quad \mathbf{v} = \mathbf{W}_v \tilde{\mathbf{h}}(v)$$

In the final dimension expansion stage, the output is further transformed by a learnable matrix $\mathbf{W}_{\text{up}}$. The complete adapter output for input $\mathbf{h}_{\text{in}}(v)$ is given in Eq. (10):

$$\alpha = \text{softmax}\left(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{\dim_{\text{head}}}}\right), \quad \text{ADP}(\mathbf{h}_{\text{in}}(v)) = \text{ReLU}(\mathbf{W}_{\text{up}}(\alpha \cdot \mathbf{v})) \quad (10)$$

5 COCO Workflow

In this section, we present the workflow of the COCO, which includes pre-training, fine-tuning, and searching phases.

5.1 Pre-training Phase

In this phase, COCO builds the HM-Index and trains the AT-GNN.

Construction for HM-Index. The HM-Index builds a hierarchical tree to predict vertex coreness in temporal graphs using a top-down approach. COCO recursively splits the time interval into smaller segments, creating a tree node for each and training a dedicated coreness prediction model. The process starts with the root node covering the full time span. For each node, a model is trained to predict coreness over its time interval. Training samples are generated by randomly selecting vertices and subintervals; for each, the true coreness is computed via core decomposition [16]. The training object is to minimize mean squared error between predicted and actual coreness values.

$$\mathcal{L}_c = \frac{1}{n_b} \sum_{i=1}^{n_b} (\hat{c}(v_i) - c(v_i))^2 \quad (11)$$

If the time interval covered by the node is sufficiently long (i.e., greater than or equal to a predefined threshold x), the interval is partitioned into p subintervals. New child nodes corresponding to these subintervals are inserted into the queue. The process repeats until no further nodes remain eligible for expansion. Once this recursive procedure completes, the HM-Index is obtained.

Training for AT-GNN. Most learning-based community search methods rely on ground-truth communities, which are unavailable for temporal graphs [22]. To overcome this, we propose an unsupervised contrastive learning approach to train AT-GNN. We design two loss functions: cohesive loss and temporal link loss.

The cohesive loss pulls embeddings of vertices likely in the same community as the query vertex V_Q closer together, while pushing apart those of unrelated vertices. A cohesive score is used to measure how strongly a vertex is connected to V_Q within the time interval T_Q .

$$\text{Cos}(v) = c(v) \cdot \frac{|\{e = (v, u, t) | u \in N(v), t \in T_Q\}|}{|N_{T_Q}(v)|} \quad (12)$$

Algorithm 2: COCO Search Algorithm

Input: A temporal graph $\mathcal{G}$, a set of query vertices V_Q , an integer r , a query time interval T_Q , HM-Index root node N_{root} , the AT-GNN $\mathcal{M}_{AT}$

Output: The resulting community C

- 1 $C \leftarrow V_Q; \{h^{(t)}(v) \mid v \in \mathcal{V}\} \leftarrow$ embeddings got by $\mathcal{M}_{AT}$;
- 2 $V_r \leftarrow r$ -hop neighbors of V_Q ;
- 3 $\tau \leftarrow \frac{\sum_{v \in V_r} \text{dis}(v, V_Q)}{|V_r - V_Q|}$; $\text{min_dis} \leftarrow 0$;
- 4 **while** $\text{min_dis} < \tau$ **do**
- 5 $C \leftarrow C \cup \arg \min_{v \in N_{T_Q}(C) \cap V_r} \text{dis}(v, V_Q)$;
- 6 $\text{min_dis} \leftarrow \min_{v \in N_{T_Q}(C) \cap V_r} \text{dis}(v, V_Q)$;
- 7 $\tau \leftarrow \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right) \cdot \tau + (1 - \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right)) \cdot \frac{\sum_{v' \in C} \text{dis}(v', T_Q)}{|C|}$;
- 8 **return** C ;

Sampling Rule 1. Cohesive loss samples consist of two parts: (1) Positive Samples V_{pos} : V_{pos} is constructed by iteratively selecting the neighbor v' with the highest $\text{CoS}(v')$ until $|V_{\text{pos}}| = |V_{\text{smp}}|$. (2) Negative Samples V_{neg} : V_{neg} is formed by randomly selecting $|V_{\text{smp}}|$ vertices from $\{\mathcal{V} - V_{\text{pos}}\}$.

After identifying positive and negative samples, we apply a margin-based triplet loss [32] for the cohesive loss. $\text{dis}(\cdot)$ denotes the average $L2$ distance between a vertex and the query vertex.

$$\mathcal{L}_{\text{Coh}} = \frac{1}{|V_{\text{pos}}| \cdot |V_{\text{neg}}|} \cdot \sum_{u \in V_{\text{pos}}} \sum_{v \in V_{\text{neg}}} \max(0, \text{dis}(u, V_Q) - \text{dis}(v, V_Q) + \epsilon) \quad (13)$$

Vertices may have varying numbers of connections. The temporal link loss aims to increase similarity between frequently connected vertices and separate those with weak links. We define a link strength function to measure connection intensity.

$$E(v, u) = \{e = (v, u, t) \mid e \in \mathcal{E}, t \in T_Q\},$$

$$\text{LinkS}(v, u) = \frac{|E(v, u)|}{\max_{(v, u, t) \in E(v, u)} t - \min_{(v, u, t) \in E(v, u)} t + 1} \quad (14)$$

Sampling Rule 2. Each temporal link loss sample $(v, v_p, v_n) \in V_{\text{temp}}$ includes three parts: (1) Anchor Vertex v : v is randomly selected from $\mathcal{V}$. (2) Positive Vertex v_p : Among the neighbors of v , the positive vertex is $v_p = \arg \max_{v' \in N_{T_Q}(v)} \text{LinkS}(v, v')$. (3) Negative Vertex v_n : Among the neighbors of v , the negative vertex is $v_n = \arg \min_{v' \in N_{T_Q}(v)} \text{LinkS}(v, v')$.

Based on the sampled triples, the temporal link loss is defined as Eq.(15). Finally, we combine the cohesive loss and the temporal link loss as Eq.(16).

$$\mathcal{L}_{\text{Temp}} = \frac{1}{|V_{\text{temp}}|} \cdot \sum_{(v_a, v_p, v_n) \in V_{\text{temp}}} \max(0, \text{dis}(v_a, v_p) - \text{dis}(v_a, v_n) + \epsilon) \quad (15)$$

$$\mathcal{L} = \mathcal{L}_{\text{Coh}} + \delta \mathcal{L}_{\text{Temp}} \quad (16)$$

5.2 Fine-tuning Phase

During fine-tuning, COCO updates only adapters of AT-GNN, minimizing overhead. Vertex coreness is computed using HM-Index, and vertices are sampled via Rule 1 and Rule 2 (Section 5.1). AT-GNN generates embeddings for these vertices to compute the loss. Unlike pre-training, where all parameters are updated, fine-tuning freezes the GNN encoder and trains only the adapter.

Table 1: Statistics of networks

Dataset	Abbr.	$ \mathcal{V} $	$ \mathcal{E} $	t_{max}
CollegeMsg	CM	1.8K	59.8K	193
MathOverflow	MO	24.8K	506.5K	2351
Superuser	SU	194.1K	1.4M	2774
WikiTalk	WT	1.1M	7.8M	2321
Youtube	YT	3.2M	9.3M	226
Flickr	FL	2.3M	33.1M	394
StackOverflow	SO	2.6M	63.5M	2774
Wikipedia	WP	6.9M	129.9M	298

5.3 Searching Phase

The search phase identifies the final community by progressively expanding the result set C , adding vertices with embeddings similar to the query vertex V_Q . We propose an adaptive search algorithm that dynamically adjusts the termination threshold τ . Guided by small-world theory [17, 27], we first extract an r -hop subgraph $G_r = (V_r, E_r)$ around V_Q . The initial threshold τ is set to the average embedding distance from V_Q to vertices in V_r , i.e., $\tau = \frac{\sum_{v \in V_r} \text{dis}(v, V_Q)}{|V_r - V_Q|}$. In each iteration, COCO adds the vertex with the smallest distance to C . After the n -th addition, τ is updated as:

$$\tau_n = \rho_n \cdot \tau_{n-1} + (1 - \rho_n) \cdot \frac{\sum_{v \in C} \text{dis}(v, V_Q)}{|C|}, \quad \rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right) \quad (17)$$

Here, $\rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right)$ smoothly adjusts τ at each step. The search stops when the minimum distance to remaining candidates exceeds the current τ .

LEMMA 5.1. *The smoothing factor $\rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right)$ decreases monotonically with community size $|C|$ and satisfies $\lim_{|C| \rightarrow |V_r|} \rho_n = 0$, guaranteeing that τ converges to $\frac{\sum_{v \in C} \text{dis}(v, V_Q)}{|C|}$.*

Our adaptive threshold offers two key advantages. First, the dynamic adjustment of τ automatically determines the search scope, eliminating the need for fixed boundaries. Second, leveraging the slope characteristics of the cosine function, the initial flat gradient maintains threshold stability to prevent premature termination, while the subsequent steep descent accelerates convergence to curb excessive expansion.

Overall, the search process is presented in Algorithm 2. The algorithm initializes the result set with the query vertex, computes all vertex embeddings via AT-GNN inference, and extracts the r -hop subgraph (lines 1-2). The initial threshold τ is set to the average distance within this subgraph (line 3). Iteratively, the closest neighbor is added to the result set, the minimal distance is updated, and τ is adaptively adjusted (lines 5-7). The search stops when the minimal distance exceeds the current threshold.

6 Performance Evaluation

All experiments were conducted on an Ubuntu server with an Intel i9-10900K CPU, 128 GB of memory, and an NVIDIA RTX 3090 GPU. Our experiments utilize eight real-world temporal networks, whose statistical properties are summarized in Table 1. More experimental results can be found in the *technical report* [1], and code is available at: <https://anonymous.4open.science/r/COCO-F183>.

Competitors. We compare COCO with seven competitors.

Table 2: Community quality comparison for different methods

Datasets	CM		MO		SU		WT		YT		FL		SO		WP		Avg +/-	
Metrics	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC
COCO	0.0064	0.5625	0.0018	0.5715	0.0011	0.5872	0.0042	0.6441	0.0065	0.6862	0.0049	0.7624	0.0097	0.5225	0.0123	0.5947	-	-
Temporal k -core	0.0044	0.6771	0.0006	0.7292	0.0001	0.6762	0.0016	0.8304	0.0008	0.7443	0.0001	0.8263	0.0031	0.7231	0.0051	0.8308	-69.75%	+23.37%
QTCS	0.0048	0.6332	0.0011	0.6572	0.0006	0.6580	0.0022	0.7408	0.0028	0.8676	0.0033	0.8695	0.0045	0.7786	OOM	OOM	-43.22%	+20.59%
TransZero	0.0026	0.8433	0.0002	0.8999	9.32 $\times 10^{-5}$	0.9293	0.0001	0.8838	0.0004	0.9650	0.0003	0.9865	7.02 $\times 10^{-5}$	0.9780	OOM	OOM	-89.14%	+51.40%
CS-Former	0.0007	0.9037	0.0003	0.9490	8.05 $\times 10^{-6}$	0.9660	0.0002	0.9388	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-94.71%	+59.24%
IACS	0.0030	0.9500	0.0002	0.8340	0.0001	0.9895	0.0004	0.6850	1.98 $\times 10^{-7}$	0.8785	8.04 $\times 10^{-8}$	0.9872	1.03 $\times 10^{-6}$	0.8942	OOM	OOM	-90.69%	+45.47%
CS-TGN	0.0024	0.9059	5.12 $\times 10^{-5}$	0.7008	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-78.64%	+41.84%
UTCS	0.0019	0.6876	0.0016	0.6678	0.0003	0.6481	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-51.90%	+16.49%

* A community is considered to be of higher quality when it exhibits a **higher TD** and a **lower TC**.

- *QTCS* [22]. A non-learning method that uses time-constrained personalized pagerank to find a β -temporal proximity core containing the query vertex.
- *Temporal k -core* [44]. A non-learning method that identifies a k -core in the projected graph of the given time interval. We employ an indexing-based method [45] to speed up its search process.
- *TransZero* [37]. An unsupervised learning framework for community search using graph transformer.
- *CSFormer* [41]. An unsupervised learning-based approach for static graphs that adds vertices whose predicted corenesses are higher than that of the query vertices into the community.
- *IACS* [7]. An inductive learning-based method for community search across different static graphs.
- *CS-TGN* [12]. A query-driven temporal GNN that captures evolving community structures using attention-based GRUs and supports interactive search via a meta-learning framework.
- *UTCS* [46]. An unsupervised method for temporal graphs that pre-trains on temporal dynamics and subgraph knowledge to identify communities using an online proximity-based scoring mechanism.

Metrics. Given the absence of ground-truth datasets for temporal graphs [22], we adopt the established evaluation framework from prior work [4, 22, 33], utilizing two widely recognized metrics: Temporal Density (TD) and Temporal Conductance (TC). For a query interval $[t_s, t_e]$ and retrieved community C , TD and TC are formally defined as follows:

$$TD(C) = \frac{2 \cdot |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v, u \in C\}|}{|C| \cdot (|C| - 1) \cdot (t_e - t_s + 1)} \quad (18)$$

$$TC(C) = \frac{|\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v \in C, u \notin C\}|}{\min\{|\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v \in C\}|, |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} | v \notin C\}|\}} \quad (19)$$

TD quantifies the internal cohesion of C by computing its average density. Higher TD values indicate stronger internal cohesion. TC measures the separation between C and the remainder of the graph. Lower TC values reflect better community isolation. Thus, an optimal community **maximizes TD** while **minimizing TC**, achieving both high internal cohesion and clear external separation.

Settings. For HM-Index construction, we set the partition coefficient $p = 4$ and leaf node threshold $x = 40$ as balanced defaults for efficiency and performance. The learning rate is fixed at 0.001. AT-GNN is trained on 200 randomly selected vertices and time intervals, with models trained for 150 epochs (with early stopping)

and fine-tuned for 10 steps. During search, we use a 5-hop subgraph around the query vertex and set $\gamma = 1$ in Eq.17. For QTCS, we use the default parameter values according to [22]. For temporal k -core, we set k to the coreness of the query vertex. For all learning-based methods, we follow the default settings provided in their papers. For methods requiring ground truth labels, such as IACS [7] and CS-TGN [12], we use the approach from TransZero [37] to generate pseudo-labels. In each experiment, we randomly generate 50 queries and report the average results. The default number of query vertices is set to 1.

6.1 Effectiveness Evaluation

Exp-1: Community Quality Comparison. We evaluate community quality for all methods using TD and TC metrics. Table 2 reports the results across all datasets. We can observe that COCO demonstrates consistent superiority, outperforming all competitors in both cohesion (TD) and separation (TC). For instance, compared with the non-learning method QTCS, COCO has +43.22% TD improvement and +20.59% TC improvement. compared with the learning method TransZero, COCO has +89.14% TD improvement, and +51.40% TC improvement. COCO’s superiority stems from its dual innovations: (1) HM-Index-derived coreness ensures cohesive community structures and (2) AT-GNN captures fine-grained evolutionary patterns. In contrast, methods such as temporal k -core and QTCS over-constrain communities via rigid k -core structures, excluding well-connected vertices. Additionally, learning-based approaches including TransZero, IACS, and CS-TGN fail to explicitly model structural cohesion, degrading performance.

Exp-2: Prediction Accuracy of HM-Index. In this experiment, we evaluate the prediction accuracy of HM-Index by predicting vertex coreness across progressive time intervals (20%, 40%, 60%, and 80% of $t_{\max}$). Results shown in Figure 5 demonstrate consistent performance above 90% accuracy across all datasets and interval lengths. Note that the accuracy is measured using $1 - \text{MAPE}$, where $\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{c(v)_i - \hat{c}(v)_i}{c(v)_i} \right|$. This robustness stems from HM-Index’s hierarchical architecture. COCO initiates its search from the root node, traverses through the HM-Index to identify nodes encompassing the query time interval. The models along the search path collectively refine coreness predictions.

6.2 Efficiency Evaluation

In efficiency experiments, we mark the algorithm as INF if it is out of memory or its running time exceeds 10^5 seconds.

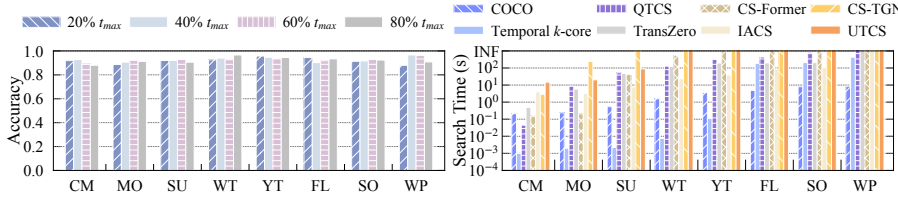


Figure 5: Prediction accuracy of HM-Index

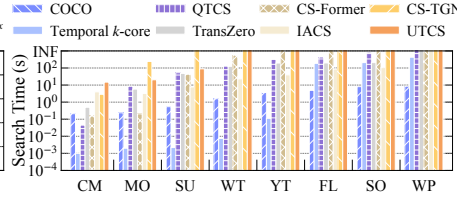
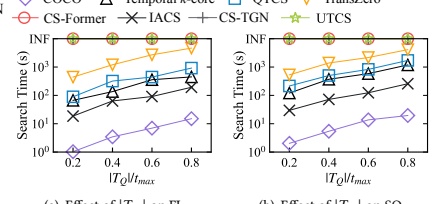
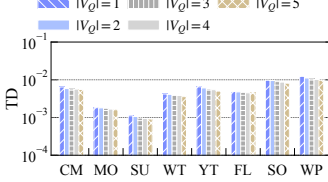
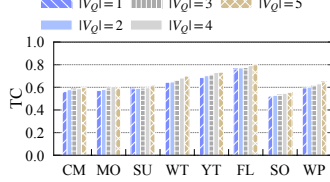
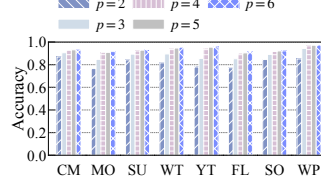
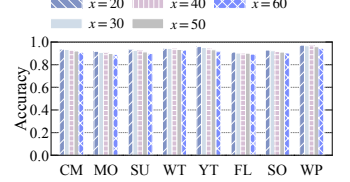
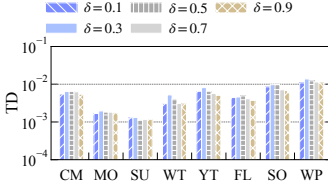
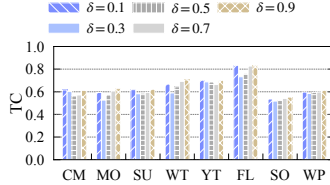
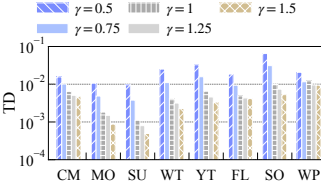
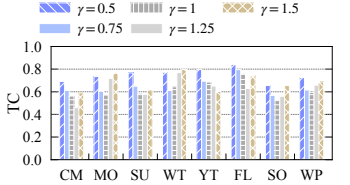


Figure 6: Search time

Figure 7: Effect of $|T_Q|$ Figure 8: Effect of $|V_Q|$ on TDFigure 9: Effect of $|V_Q|$ on TCFigure 10: Effect of p on accuracyFigure 11: Effect of x on accuracyFigure 12: Effect of δ on TDFigure 13: Effect of δ on TCFigure 14: Effect of γ on TDFigure 15: Effect of γ on TC

Exp-3: Overall Search Time Comparison. Compared to learning-based methods, COCO achieves significantly higher speed, thanks to its efficient fine-tuning mechanism. Compared to non-learning methods, COCO demonstrates (1) being approximately two orders of magnitude faster than QTCs, and (2) variable performance against temporal k -core: slower on small temporal graphs (i.e., CM, MO, and SU datasets) but faster on large temporal graphs (i.e., FL, SO, and WP datasets). This is because the temporal k -core index can be constructed on small temporal graphs, where temporal k -core can be directly retrieved. However, for large temporal graphs, the temporal k -core index cannot be constructed and requires online decomposition, degrading the search time.

Exp-4: Effect of $|T_Q|$ on Search Time. We investigate the impact of query time interval length $|T_Q|$ on search efficiency, testing query time intervals spanning 20-80% of $t_{\max}$. Figure 7 reports the results on FL and SO datasets. We have two key findings. First, all methods exhibit more search times with longer query time intervals, as expanding temporal ranges require processing more vertices/edges. Second, COCO consistently outperforms baselines by at least one order of magnitude due to its efficient fine-tuning mechanism that reduces computational overhead.

6.3 Parameters Analysis

We analyze COCO's parameter sensitivity by varying several key parameters. For the query setting, we vary the number of query vertices, denoted as $|V_Q|$. For the HM-Index, we adjust two parameters: the partition coefficient p and the leaf node threshold x . For AT-GNN, we consider two additional parameters: δ , which adjusts the loss weights in Eq. (16), and γ , which controls the search process in Eq. (17).

Exp-5: Effect of $|V_Q|$. The effect of the number of query vertices on community quality is shown in Figures 8 and 9. As $|V_Q|$ increases,

we observe a slight downward trend in TD and a gradual upward trend in TC across all datasets. Larger query sets typically require more intermediate vertices to maintain structural connectivity and cohesiveness, expanding community size. This slightly reduces internal interaction density and weakens external separation. Despite this, COCO maintains stable performance across query scales, demonstrating its robustness in multi-vertex community search.

Exp-6: Effect of p . The effect of the partition coefficient p on the prediction accuracy of HM-Index is shown in Figure 10. As observed in the figure, the prediction accuracy shows a consistent upward trend as p increases across all datasets. This improvement is primarily because a larger partition coefficient leads to more branches in the hierarchical tree, allowing for a finer-grained division of the time intervals. Such detailed partitioning enables the lightweight regression models at each node to better capture subtle temporal dynamics and structural properties within specific sub-intervals, thereby refining the coreness estimation. While larger p values enhance accuracy, $p = 4$ is chosen as the default setting to maintain an optimal balance between prediction reliability and the structural complexity of the index.

Exp-6: Effect of x . The effect of the leaf node threshold x , used when constructing the HM-Index, on prediction accuracy is shown in Figure 11. As shown in the figure, prediction accuracy slightly declines as x increases from 20 to 60 across all datasets. A smaller x yields finer temporal partitions in the HM-Index leaf nodes, allowing regression models to better capture short-term dynamics and structural variations in vertex coreness. In contrast, larger x leads to coarser intervals that may miss subtle temporal patterns, causing a minor accuracy drop. Nevertheless, accuracy remains above 90% for all x values. We choose $x = 40$ as the default to balance prediction performance and indexing efficiency.

Table 3: Ablation study

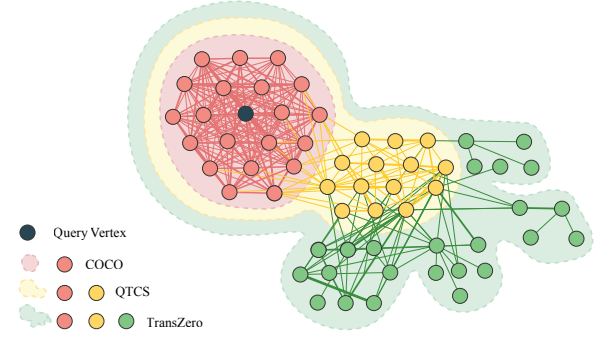
Datasets	CM		MO		SU		WT		YT		FL		SO		WP		Avg +/-	
Metrics	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC	TD	TC
COCO (full)	0.0064	0.5625	0.0018	0.5715	0.0011	0.5872	0.0042	0.6441	0.0065	0.6862	0.0049	0.7624	0.0097	0.5225	0.0123	0.5947	-	-
w/o HM-Index (use single MLP)	0.0054	0.7838	0.0017	0.7358	0.0009	0.7232	0.0035	0.8032	0.0044	0.9043	0.0031	0.8025	0.0080	0.6567	0.0083	0.7354	-22.06%	+25.29%
w/o AT-GNN (use GAT)	0.0051	0.5935	0.0015	0.7685	0.0010	0.5953	0.0038	0.7887	0.0057	0.7803	0.0046	0.8742	0.0093	0.5777	0.0097	0.7629	-12.15%	+16.17%
w/o Adapter (use linear layer)	0.0057	0.6208	0.0016	0.6522	0.0011	0.6037	0.0038	0.6491	0.0065	0.6943	0.0048	0.8639	0.0084	0.5237	0.0110	0.6355	-7.52%	+6.21%
w/o DFT (use raw temporal feature)	0.0051	0.5935	0.0015	0.7685	0.0010	0.5953	0.0038	0.7887	0.0057	0.7803	0.0046	0.8742	0.0093	0.5777	0.0097	0.7629	-12.15%	+16.17%
w/o $\mathcal{L}_{Coh}$ (use $\mathcal{L}_{Temp}$)	0.0041	0.5856	0.0017	0.5959	0.0009	0.7158	0.0036	0.7176	0.0041	0.7662	0.0041	0.8366	0.0066	0.6392	0.0113	0.5952	-20.75%	+10.69%
w/o $\mathcal{L}_{Temp}$ (use $\mathcal{L}_{Coh}$)	0.0056	0.6655	0.0017	0.6958	0.0009	0.6470	0.0035	0.7731	0.0059	0.6938	0.0039	0.9249	0.0082	0.5892	0.0118	0.7635	-12.21%	+16.73%
w/o Coreness (use Node2Vec)	0.0063	0.5952	0.0018	0.5675	0.0010	0.6358	0.0039	0.6739	0.0063	0.7348	0.0045	0.8115	0.0091	0.5473	0.0118	0.6139	-5.09%	+5.16%

Exp-8: Effect of δ . We evaluate the impact of δ , which balances cohesive loss and temporal link loss in Eq.(16), across values $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. Figures 12 and 13 present the results. We observe an initial quality improvement (rising TD, falling TC) as δ increases to 0.5, followed by degradation at higher values. It is because excessive temporal loss weighting ($\delta > 0.5$) undermines structural cohesion. The results establish $\delta = 0.5$ as the optimal trade-off, simultaneously maximizing community cohesiveness and temporal consistency across most datasets.

Exp-9: Effect of γ . We study the effect of γ that controls the threshold decay rate in the search function (Eq.(17)) by varying values from 0.5 to 1.5. The results shown in Figures 14 and 15 reveal that increasing γ reduces TD by producing larger communities, while causes minor TC fluctuations due to community size’s limited impact on TC. Notably, COCO maintains robust performance even with the default $\gamma = 1$, demonstrating its stability across parameter choices.

6.4 Ablation Study

Exp-10: Ablation Study. We evaluate key components of COCO, including the hierarchical model index, the AT-GNN (including its adapter and frequency modeling), the loss functions, and coreness usage¹¹. Results are summarized in Table 3. (1) Replacing the HM-Index with a single MLP leads to significant performance degradation, particularly a 22.06% average drop in TD, confirming the necessity of the hierarchical structure for coreness estimation. (2) Substituting AT-GNN with GAT results in a quality drop of 12.15% in TD and 16.17% in TC, demonstrating AT-GNN’s superior capability in capturing temporal community patterns. (3) To verify the adaptive mechanism, we replace the entire adapter module with a single linear layer, which effectively removes the query-context features (s_Q) and the bottleneck architecture; the resulting decline in community quality proves that simple dimension transformation cannot handle query-specific graph dynamics. (4) Utilizing raw temporal features instead of DFT modeling leads to performance similar to the GAT variant, underscoring the importance of frequency-domain features for capturing interaction patterns. (5) Removing the cohesive loss $\mathcal{L}_{Coh}$ yields an average TD drop of 20.75%, while (6) excluding the temporal link loss $\mathcal{L}_{Temp}$ leads to a 12.21% TD decrease and 16.73% TC increase, validating that both functions are vital for maintaining structural and temporal consistency. (7) Finally, replacing structural coreness with Node2Vec features degrades community quality, indicating that coreness is a more effective indicator for identifying cohesive subgraphs. Overall,

**Figure 16: Case study on DBLP**

all proposed components contribute meaningfully to the performance of COCO.

6.5 Case Study

Exp-11: Case Study. We conduct a case study on a temporal co-authorship network from DBLP [5], where vertices represent authors and timestamped edges denote publication collaborations. In the case study, we set the query time interval from 2020 to 2025. Figure 16 visualizes communities identified by COCO, QTCS, and TransZero, with edge thickness reflecting interaction frequency and the query author marked in black. COCO’s community, which consists of red vertices, demonstrates superior cohesion, exhibiting: (1) consistently high interaction frequencies among members, forming a tight collaboration network; (2) precise exclusion of weakly-connected authors (unlike QTCS’s yellow vertices with limited query author ties); and (3) avoidance of sparsely-interacting groups (unlike TransZero’s green vertices with few interactions). This real-world validation confirms COCO’s ability to identify authentic, tightly-knit academic communities.

7 Conclusion

This paper addresses the problem of time-interval-based community search. We propose COCO - a novel cohesiveness-aware learning framework that synergistically combines graph neural networks with structural constraints. COCO’s three-phase architecture (pre-training, fine-tuning, and searching) innovatively integrates two key components: (1) the HM-Index for hierarchical coreness prediction, and (2) AT-GNN for adaptive temporal pattern learning. COCO achieves both high-quality community finding and computational efficiency, as demonstrated through extensive experiments on real-world datasets.

References

- [1] Anonymous Author(s). 2026. COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs (Technical Report). <https://anonymous.4open.science/r/COCO-F183/COCO-SIGIR26-Technical%20Report.pdf>
- [2] Ali Behrouz, Farnoosh Hashemi, and Laks VS Lakshmanan. 2022. FirmTruss community search in multilayer networks. *Proceedings of the VLDB Endowment* 16, 3 (2022), 505–518.
- [3] Xueqin Chang, Qing Liu, Yunjun Gao, Baihua Zheng, Yi Cai, and Qing Li. 2025. The most influenced community search on social networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2601–2614.
- [4] Lingyang Chu, Yanyan Zhang, Yu Yang, Lanjun Wang, and Jian Pei. 2019. Online density bursting subgraph detection from temporal graphs. *Proceedings of the VLDB Endowment* 12, 13 (2019), 2353–2365.
- [5] DBLP. 2024. <https://dblp.uni-trier.de/xml/>
- [6] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2022. Delta tuning: A comprehensive study of parameter efficient methods for pre-trained language models. *arXiv preprint arXiv:2203.06904* (2022).
- [7] Shuheng Fang, Kangfei Zhao, Yu Rong, Zhixun Li, and Jeffrey Xu Yu. 2024. Inductive attributed community search: to learn communities across graphs. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2576–2589.
- [8] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29 (2020), 353–392.
- [9] Yixiang Fang, Zhongran Wang, Reynold Cheng, Hongzhi Wang, and Jiafeng Hu. 2018. Effective and efficient community search over large directed graphs. *IEEE Transactions on Knowledge and Data Engineering* 31, 11 (2018), 2093–2107.
- [10] Edoardo Galimberti, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2018. Mining (maximal) span-cores from temporal networks. In *Proceedings of the 27th ACM international Conference on Information and Knowledge Management*. 107–116.
- [11] Edoardo Galimberti, Martino Ciaperoni, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2020. Span-core decomposition for temporal networks: Algorithms and applications. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15, 1 (2020), 1–44.
- [12] Farnoosh Hashemi, Ali Behrouz, and Milad Rezaei Hajidehi. 2023. Cs-tgn: Community search via temporal graph neural networks. In *Companion Proceedings of the ACM Web Conference 2023*. 1196–1203.
- [13] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*. PMLR, 2790–2799.
- [14] Yuli Jiang, Yu Rong, Hong Cheng, Xin Huang, Kangfei Zhao, and Junzhou Huang. 2022. Query driven-graph neural networks for community search: From non-attributed, attributed, to interactive attributed. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1243–1255.
- [15] Zongli Jiang, Yirui Tan, Guoxin Chen, Fangda Guo, Jinli Zhang, and Xiaolu Bai. 2024. GNN-Based Persistent K-core Community Search in Temporal Graphs. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 569–578.
- [16] Wissam Khaoiuid, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proceedings of the VLDB Endowment* 9, 1 (2015), 13–23.
- [17] Jon M Kleinberg. 2000. Navigation in a small world. *Nature* 406, 6798 (2000), 845–845.
- [18] Ling Li, Siqiang Luo, Yuhai Zhao, Caihua Shan, Zhengkui Wang, and Lu Qin. 2023. COCLEP: Contrastive learning-based semi-supervised community search. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2483–2495.
- [19] Ling Li, Yuhai Zhao, Yuan Li, Fazal Wahab, and Zhengkui Wang. 2022. The most active community search in large temporal graphs. *Knowledge-Based Systems* 250 (2022), 109101.
- [20] Rong-Hua Li, Jiao Su, Lu Qin, Jeffrey Xu Yu, and Qiangqiang Dai. 2018. Persistent community search in temporal networks. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 797–808.
- [21] Yuan Li, Jinsheng Liu, Huiqun Zhao, Jing Sun, Yuhai Zhao, and Guoren Wang. 2021. Efficient continual cohesive subgraph search in large temporal graphs. *World Wide Web* 24 (2021), 1483–1509.
- [22] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Chunxue Zhu, Hongchao Qin, Hai Jin, and Tao Jia. 2024. QTCs: Efficient Query-Centered Temporal Community Search. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1187–1199.
- [23] Qing Liu, Xuankun Liao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2023. Distributed (α, β) -core decomposition over bipartite graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 909–921.
- [24] Qing Liu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. Truss-based community search over large directed graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2183–2197.
- [25] Qing Liu, Yifan Zhu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. VAC: vertex-centric attributed community search. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 937–948.
- [26] Chengyang Luo, Qing Liu, Yunjun Gao, and Jianliang Xu. 2025. Synergetic Community Search over Large Multilayer Graphs. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1412–1424.
- [27] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [28] Fragkiskos D Malliaros, Christos Giatsidis, Apostolos N Papadopoulos, and Michalis Vazirgiannis. 2020. The core decomposition of networks: Theory, algorithms and applications. *The VLDB Journal* 29, 1 (2020), 61–92.
- [29] Hongchao Qin, Rong-Hua Li, Guoren Wang, Lu Qin, Yurong Cheng, and Ye Yuan. 2019. Mining periodic cliques in temporal networks. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1130–1141.
- [30] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Lu Qin, and Zhiwei Zhang. 2022. Mining bursting core in large temporal graphs. *Proceedings of the VLDB Endowment* (2022).
- [31] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Weihua Yang, and Lu Qin. 2020. Periodic communities mining in temporal networks: Concepts and algorithms. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3927–3945.
- [32] Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 815–823.
- [33] Arlei Silva, Ambuj Singh, and Ananthram Swami. 2018. Spectral algorithms for temporal graph cuts. In *Proceedings of the 2018 World Wide Web Conference*. 519–528.
- [34] Qing Sima, Jianke Yu, Xiaoyang Wang, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2025. Deep Overlapping Community Search via Subspace Embedding. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
- [35] Longxu Sun, Xin Huang, Zheng Wu, and Jianliang Xu. 2024. Efficient Cross-layer Community Search in Large Multilayer Graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2959–2971.
- [36] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [37] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Efficient unsupervised community search with pre-trained graph transformer. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2227–2240.
- [38] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Billion attributed community search at billion scale. *Proceedings of the ACM on Management of Data* 1, 4 (2024), 1–25.
- [39] Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Efficient and effective community search on large-scale bipartite graphs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 85–96.
- [40] Kai Wang, Wenjie Zhang, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Discovering significant communities on bipartite graphs: An index-based approach. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2021), 2471–2485.
- [41] Yuxiang Wang, Xiaoxuan Gou, Xiaoliang Xu, Yuxia Geng, Xiangyu Ke, Tianxing Wu, Zhiyuan Yu, Runhui Chen, and Xiangyang Wu. 2024. Scalable community search over large-scale graphs based on graph transformer. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1680–1690.
- [42] Yuxiang Wang, Shuzhan Ye, Xiaoliang Xu, Yuxia Geng, Zhenghe Zhao, Xiangyu Ke, and Tianxing Wu. 2024. Scalable community search with accuracy guarantee on attributed graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2737–2750.
- [43] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2023. Scalable time-range k-core query on temporal graphs (full version). *arXiv preprint arXiv:2301.03770* (2023).
- [44] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. Evolution Forest Index: Towards Optimal Temporal k-Core Component Search via Time-Topology Isomorphic Computation. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2840–2853.
- [45] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On querying historical k-cores. *Proceedings of the VLDB Endowment* (2021).
- [46] Yue Zhang, Yankai Chen, Yingli Zhou, Yucan Guo, Xiaolin Han, and Chenhao Ma. 2025. UTCS: Effective Unsupervised Temporal Community Search with Pre-training of Temporal Dynamics and Subgraph Knowledge. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2647–2651.
- [47] Yifei Zhang, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Significant engagement community search on temporal networks. In *International Conference on Database Systems for Advanced Applications*. Springer, 250–258.
- [48] Ming Zhong, Junyong Yang, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. A Unified and Scalable Algorithm Framework of User-Defined Temporal (k, X) -Core Query. *IEEE Transactions on Knowledge and Data Engineering* (2024).

A Proof of Lemma 5.1

PROOF. As $|C|$ increases from $|V_Q|$ to $|V_r|^\gamma$, $\frac{\pi}{2} \cdot \frac{|C|}{|V_r|^\gamma}$ grows from $\frac{\pi}{2} \cdot \frac{|V_Q|}{|V_r|^\gamma}$ to $\frac{\pi}{2}$. Within this interval, the cosine function is strictly decreasing, which means ρ_n strictly decreases and approaches 0. In the update equation, when ρ_n approaches 0, the term $\rho_n \cdot \tau_{n-1}$ becomes negligible while $(1 - \rho_n)$ tends toward 1. Consequently, τ_n becomes increasingly determined by the current average distance $\frac{\sum_{v \in C} \text{dis}(v, V_Q)}{|C|}$. Since vertices are added in order of increasing distance, this average distance is non-decreasing and eventually stabilizes, causing τ to converge to this stabilized average distance. $\square$

B Time Complexity of Different Phases of COCO

Pre-training Phase The time complexity contains two components. First, the HM-Index tree construction involves building a tree structure with depth $h = \log_p \frac{t_{\max}}{x}$, where $t_{\max}$ is the maximal timestamp, x the leaf interval length, and p the branching factor. The core decomposition process requires $O(|V| + |E|)$ time in total across all timestamps. The HM-Index structure contains $O(\frac{t_{\max}}{x})$ nodes. For each node, the training process requires $O(|V| \cdot \log \frac{t_{\max}}{x})$ time per epoch. This leads to an overall time complexity of $O(|E| + \lambda \cdot \frac{t_{\max}}{x} \cdot |V| \cdot \log \frac{t_{\max}}{x})$. Second, the AT-GNN processes K GNN layers with feature dimension d . Each layer requires $O(|V| + |E|)$ time. Training on Q queries, this results in a time complexity of $O(\lambda \cdot Q \cdot K \cdot (|V| + |E|))$. Overall, the total time complexity is $O(\lambda \cdot \frac{t_{\max}}{x} \cdot |V| \cdot \log \frac{t_{\max}}{x} + \lambda \cdot Q \cdot K \cdot (|V| + |E|))$.

Fine-tuning Phase. The computational cost in this phase is dominated by updating the parameters in the adapters, while the encoder parameters remain frozen. Consequently, fine-tuning the adapters requires $O(\lambda \cdot Q \cdot K \cdot |V|)$ time.

Searching Phase. The process begins with predicting vertex corenesses in the r -hop subgraph using HM-Index, which requires $O(|V_r| \cdot \log \frac{t_{\max}}{x})$ time. Next, vertex embeddings are generated by AT-GNN and subsequently sorted, taking $O(K \cdot (|V_r| + |E_r|) + |V_r| \cdot \log |V_r|)$ time. Finally, the search process completes in $O(|V_r|)$ iterations. Therefore, the overall time complexity is $O(K \cdot (|V_r| + |E_r|) + |V_r| \cdot (\log |V_r| + \log \frac{t_{\max}}{x}))$.

C Analysis of the Prediction Accuracy of HM-Index

LEMMA C.1. *The HM-Index prediction operates as a recursive residual learning process, where the parent prediction $\hat{c}_{j-1}$ serves as a structural prior refined by local temporal features S_j . This mechanism induces a contractive error bound at depth j , given by $\epsilon_j \leq \beta \epsilon_{j-1} + \delta$, where $\beta \in [0, 1)$ is the residual refinement coefficient. Consequently, the cumulative error at leaf depth h does not explode but converges to a bounded limit:*

$$\epsilon_h \leq \beta^h \epsilon_0 + \frac{\delta(1 - \beta^h)}{1 - \beta}$$

PROOF. The HM-Index functions as a coarse-to-fine residual learning chain, where the model $\mathcal{M}_j$ at depth j treats the parent's prediction $\hat{c}_{j-1}$ as a structural baseline rather than predicting coreness in isolation. By utilizing fine-grained local features S_j to learn a correction term, formulated as $\hat{c}_j \approx \hat{c}_{j-1} + \Delta(S_j)$, the model transforms the task into local residual estimation. This approach induces

a contraction property on the absolute error ϵ_j : the refinement coefficient $\beta < 1$ ensures that inherited deviations are dampened at each level, subject only to an irreducible local approximation error δ .

Solving the recurrence relation $\epsilon_j \leq \beta \epsilon_{j-1} + \delta$ recursively from the leaf node back to the root reveals the stability of this architecture. The expansion $\epsilon_h \leq \beta^h \epsilon_0 + \delta \sum_{i=0}^{h-1} \beta^i$ demonstrates that the initial propagated error decays exponentially with depth. Meanwhile, the accumulation of local residuals forms a geometric series that converges to a finite constant. This proves that increasing the hierarchical depth prevents error explosion, as the final error is strictly bounded by the model's local capacity and refinement efficiency. $\square$

D Proof of Theorem 4.1

PROOF. Let $t_{\max}$ be the maximum timestamp of the graph, and x be the maximum length of the time interval for the leaf nodes in the HM-Index. The number of leaf nodes in HM-Index is bounded by $O(\frac{t_{\max}}{x})$. Since HM-Index is a balanced tree, the total number of nodes is also bounded by $O(\frac{t_{\max}}{x})$. The height of HM-Index is $h = \log_p \frac{t_{\max}}{x}$. In the worst case, Algorithm 1 traverses the HM-Index from the root to the leaf node. Hence, the time complexity of Algorithm 1 is bounded by $O(\log \frac{t_{\max}}{x})$. Each node contains a model and a set of corenesses for vertices over the corresponding time interval. Therefore, the space required for a single node is bounded by $O(|M| + |V|)$. Consequently, the overall space complexity of HM-Index is bounded by $O(\frac{t_{\max}}{x} \cdot (|M| + |V|))$.

Solving the recurrence relation $\epsilon_j \leq \beta \epsilon_{j-1} + \delta$ recursively from the leaf node back to the root reveals the stability of this architecture. The expansion $\epsilon_h \leq \beta^h \epsilon_0 + \delta \sum_{i=0}^{h-1} \beta^i$ demonstrates that the initial propagated error decays exponentially with depth. Meanwhile, the accumulation of local residuals forms a geometric series that converges to a finite constant. This proves that increasing the hierarchical depth prevents error explosion, as the final error is strictly bounded by the model's local capacity and refinement efficiency. $\square$

E Pseudocode for Forward Propagation of AT-GNN

As shown in Algorithm 3, forward propagation of AT-GNN begins by computing initial embeddings for vertices and edges (lines 1–2). Then, the model performs K layers of message passing. Each layer includes a pre-aggregation adapter, an encoder, and a post-aggregation adapter. In the pre-aggregation adapter, the input embedding $\mathbf{h}^{(t-1)}(v)$ is transformed into an intermediate representation $\mathbf{h}_{\text{pre}}(v)$ using the ADP procedure (line 5). ADP improves embeddings by adding query-specific context. It builds a context vector $\mathbf{s}_Q$ from the query vertex set V_Q and its neighbors, applies self-attention, and outputs a refined representation (lines 14–18). Next, the encoder aggregates neighbor information. For each neighbor v' of v , attention scores $\alpha_{v,v'}$ are calculated using channel attention, and used to combine neighbor embeddings (lines 6–10). The result, $\mathbf{h}'(v)$, is further processed by the post-aggregation adapter via ADP (line 11). Finally, the output combines features from all three components (line 12).

Algorithm 3: Forward Propagation of AT-GNN

Input: A temporal graph $\mathcal{G}$, a set of query vertices V_Q , a query time interval T_Q
Output: The embeddings of $\mathcal{V}$

```

1  $\{\mathbf{h}^{(0)}(v) \leftarrow f_v([q(v)||s_c(v)]) \mid v \in \mathcal{V}\}$  // Eq. (4)
2  $\{I_{v,v'} \leftarrow [\text{DFT}(f_e([a_t(v,v')])_{t=t_s}^{t_e})]_{0:\text{dim}_e} \mid e = (v,v'), t \in \mathcal{E}, t \in T_Q\}$  // Eq. (5)
3 for  $l = 1$  to  $K$  do
4   for  $v \in \mathcal{V}$  do
5      $\mathbf{h}_{\text{pre}}(v) \leftarrow \text{ADP}(\mathbf{h}^{(l-1)}(v))$ ; // Pre-aggregation adapter
6     // encoder Eq. (6) to (8)
7     for  $v' \in \{v\} \cup N_{T_Q}(v)$  do
8        $\mathbf{x}_{v,v'} \leftarrow \sigma(\text{MLP}(I_{v,v'}))$ ;  $\mathbf{l}'_{v,v'} \leftarrow \mathbf{l}_{v,v'} \odot \mathbf{x}_{v,v'}$ ;
9        $b_{v,v'} = \text{LeakyReLU}(\mathbf{a}^T \cdot [\mathbf{W}_v \mathbf{h}^{(l-1)}(v) || \mathbf{W}_{v'} \mathbf{h}^{(l-1)}(v') || \mathbf{l}'_{v,v'}])$ ;
10       $\alpha_{v,v'} = \text{softmax}_{v'}(b_{v,v'})$ ;
11       $\mathbf{h}'(v) \leftarrow \sum_{v' \in N(v) \cup \{v\}} \alpha_{v,v'} \cdot \mathbf{h}^{(l-1)}(v')$ ;
12       $\mathbf{h}_{\text{post}}(v) \leftarrow \text{ADP}(\mathbf{h}'(v))$ ; // Post-aggregation adapter
13       $\mathbf{h}^{(l)} \leftarrow \mathbf{h}_{\text{pre}}(v) + \mathbf{h}'(v) + \mathbf{h}_{\text{post}}(v)$ ;
14 return  $\{\mathbf{h}^{(l)}(v) \mid v \in \mathcal{V}\}$ ;

Procedure  $\text{ADP}(\mathbf{h}_{\text{in}}(v))$ 
15  $s_Q \leftarrow [\bar{c}(V_Q) || \bar{c}(N(V_Q))]_{\frac{t_s}{t_{\max}}}^{\frac{t_e}{t_{\max}}}$ ;
16  $\tilde{\mathbf{h}}(v) \leftarrow [\mathbf{W}_{\text{down1}} \mathbf{h}_{\text{in}}(v) || \mathbf{W}_{\text{down2}} s_Q]$ ; // Eq. (9)
17  $\mathbf{q} \leftarrow \mathbf{W}_q \tilde{\mathbf{h}}(v)$ ;  $\mathbf{k} \leftarrow \mathbf{W}_k \tilde{\mathbf{h}}(v)$ ;  $\mathbf{v} \leftarrow \mathbf{W}_v \tilde{\mathbf{h}}(v)$ ; // Eq. (9)
18  $\alpha \leftarrow \text{softmax}(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{\text{dim}_{\text{head}}}})$ ; // Eq. (10)
19 return  $\text{ReLU}(\mathbf{W}_{\text{up}}(\alpha \cdot \mathbf{v}))$ ; // Eq. (10)

```

F Pseudocode for Construction of HM-Index**Algorithm 4: HM-Index Construction**

Input: A temporal graph $\mathcal{G}$, a partition coefficient p , a length threshold x , a learning rate φ , a number of epochs λ , a batch size n_b
Output: Root node of HM-Index

```

1 Initialize queue  $L$  with a node  $N_{\text{root}}$  with entire time interval  $\mathcal{T}$ ;
2 while  $L \neq \emptyset$  do
3    $N \leftarrow L.\text{dequeue}()$ ; initialize a model  $M_N$  for  $N$ ;
4   Sample batches  $\{V_1, V_2, \dots, V_n\}$  where each  $|V_{b_i}| = n_b$ ;
5   for  $\text{epoch} = 1$  to  $\lambda$  do
6     for  $\text{each batch } V_i \in \{V_1, V_2, \dots, V_n\}$  do
7        $\mathcal{L}_c \leftarrow 0$ ;
8       for  $v \in V_i$  do
9          $[t_s, t_e] \leftarrow$  randomly sample subinterval from  $T_N$ ;
10         $c(v)_{[t_s, t_e]} \leftarrow$  compute the coreness of  $v$ ;
11         $\hat{c}_{[t_s, t_e]}(v) \leftarrow$  predict using Algorithm 1;
12         $\mathcal{L}_c \leftarrow \mathcal{L}_c + \frac{1}{n_b} (\hat{c}(v) - c(v))^2$ ;
13       $\theta(M_N) \leftarrow \theta(M_N) - \varphi \cdot \nabla_{\theta} \mathcal{L}_c$ ;
14   if  $|T_N| > x$  then
15     Create  $p$  child nodes with equal-length subintervals;
16     Enqueue child nodes into  $L$ ;
17 return  $N_{\text{root}}$ ;

```

As outlined in Algorithm 4, the procedure starts by initializing a queue L containing the root node. The root node covers the entire time interval $\mathcal{T}$ (line 1). For each node N from the queue, the algorithm first creates a model (line 4) and then randomly samples batches of vertices (line 5). For each vertex v in the batch, it selects a random subinterval of T_N (line 9). The coreness of v over this subinterval is computed using the core decomposition method [16] and serves as the ground-truth label (line 10). Meanwhile, Algorithm 1 generates the predicted coreness of v (line 11). The training objective is defined as the mean squared error between the predicted and actual values.

G Pseudocode for Pre-training of AT-GNN**Algorithm 5: AT-GNN Pre-training**

Input: A temporal graph $\mathcal{G}$, a learning rate φ , a number of epochs λ , a batch size n_b , a initialized AT-GNN model M_{AT}
Output: The trained model M_{AT}

```

1 Generate queries batches  $\{Q_1, Q_2, \dots, Q_n\}$  where  $Q_i$  has  $n_b$  queries;
2 for  $\text{epoch} = 1$  to  $\lambda$  do
3   for  $\text{each batch } Q_i \in \{Q_1, Q_2, \dots, Q_n\}$  do
4      $\mathcal{L} \leftarrow 0$ ;
5     for  $\text{each query } (V_Q, T_Q) \in Q_i$  do
6        $\{\mathbf{h}^{(l)}(v) \mid v \in \mathcal{V}\} \leftarrow$  embeddings got by  $M_{\text{AT}}$ ;
7        $V_{\text{pos}}, V_{\text{neg}} \leftarrow$  sample by rule 1;
8        $\mathcal{L}_{\text{Coh}} \leftarrow \frac{1}{n_b} \cdot \frac{1}{|V_{\text{pos}}| \cdot |V_{\text{neg}}|} \sum_{u \in V_{\text{pos}}} \sum_{v \in V_{\text{neg}}} \max(0, \text{dis}(u, V_Q) - \text{dis}(v, V_Q) + \epsilon)$ ;
9        $V_{\text{temp}} \leftarrow$  sample by rule 2;
10       $\mathcal{L}_{\text{Temp}} \leftarrow \frac{1}{n_b} \cdot \frac{1}{|V_{\text{temp}}|} \sum_{(v_a, v_p, v_n) \in V_{\text{temp}}} \max(0, \text{dis}(v_a, v_p) - \text{dis}(v_a, v_n) + \epsilon)$ ;
11       $\mathcal{L} \leftarrow \mathcal{L}_{\text{Coh}} + \delta \mathcal{L}_{\text{Temp}}$ ;
12       $\theta(M_{\text{AT}}) \leftarrow \theta(M_{\text{AT}}) - \varphi \cdot \nabla_{\theta} \mathcal{L}$ ;
13 return  $M_{\text{AT}}$ ;

```

The overall training process of AT-GNN is presented in Algorithm 5. The algorithm starts by generating several query batches (line 1). Each query has a different query vertex and a different time interval. This allows AT-GNN to learn community information across various time intervals. For each query, the algorithm samples data for the cohesive loss and computes the loss (lines 7-8). It then samples data for the temporal link loss and computes the corresponding loss (lines 9-10). These two losses are combined (line 11). After processing each batch, the parameters of AT-GNN are updated (line 12).

H Pseudocode for Fine-tuning of At-GNN

The process of fine-tuning is shown in Algorithm 6. First, COCO gets the coreness of all vertices using HM-Index via Algorithm 3 (line 1). Then it samples vertices based on Rule 1 and Rule 2 from Section 5.1 (lines 2). For each fine-tuning step, COCO generates the vertex embeddings using AT-GNN (line 4). Next, sampled vertices are used to compute the loss (line 5). The key difference between

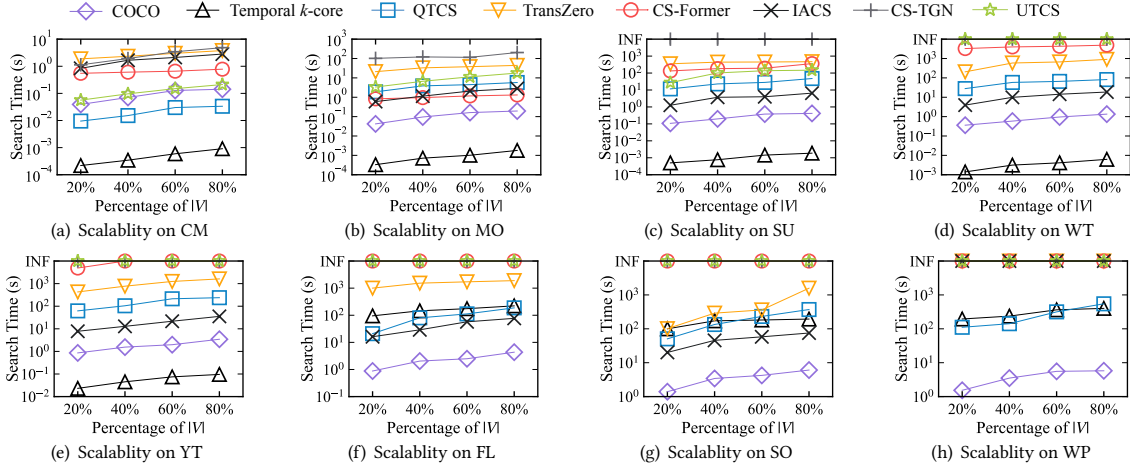


Figure 17: Scalability

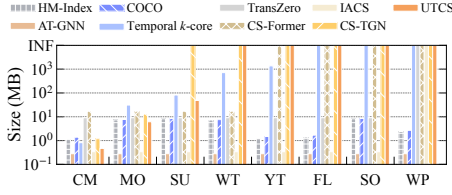


Figure 18: Model/Index size

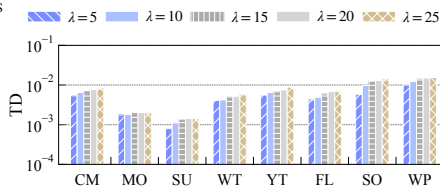


Figure 19: Effect of # of fine-tuning steps on TD

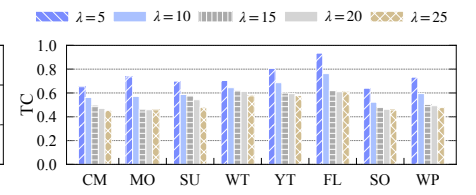


Figure 20: Effect of # of fine-tuning steps on TC

Algorithm 6: COCO Fine Tuning

Input: A temporal graph $\mathcal{G}$, a set of query vertices' V_Q , a query time interval T_Q , a learning rate φ , a number of fine tuning λ , the AT-GNN $\mathcal{M}_{AT}$ model, adapter modules $\mathcal{M}_{ADP}$, HM-Index root node N_{root}

Output: The fine-tuned AT-GNN

- 1 $\{\hat{c}(v) | v \in \mathcal{V}\} \leftarrow$ predict corenesses using N_{root} via Algorithm 1;
- 2 $V_{pos}, V_{neg}, V_{temp} \leftarrow$ sample by rules 1 and 2;
- 3 **for** $epoch = 1$ **to** λ **do**
- 4 $\mathcal{L} \leftarrow 0$; $\{\mathbf{h}^{(l)}(v) | v \in \mathcal{V}\} \leftarrow$ infer with $\mathcal{M}_{AT}$ via Algorithm 3;
- 5 $\mathcal{L} \leftarrow$ compute the loss function based on $V_{pos}, V_{neg}, V_{temp}$;
- 6 $\theta(\mathcal{M}_{ADP}) \leftarrow \theta(\mathcal{M}_{ADP}) - \varphi \cdot \nabla_{\theta} \mathcal{L}$;
- 7 **return** $\mathcal{M}_{AT}$

pre-training and fine-tuning lies in parameter updates. During pre-training, all parameters in AT-GNN are updated. However, during fine-tuning, the parameters in the GNN encoder are frozen. Only the parameters in the adapter module are updated based on the computed loss (line 6).

I Details on Experimental Metrics and Settings

For QTCS, we use the default parameter values according to [22]. For temporal k -core, we set k to the coreness of the query vertex. For all learning-based methods, we follow the default settings provided in their papers. For methods requiring ground truth labels, such as IACS [7] and CS-TGN [12], we use the approach from TransZero [37] to generate pseudo-labels.

J More Experimental Results

Exp-12: Model/Index Size. In figure 18, we can observe that COCO's AT-GNN achieves a more compact architecture than competing learning-based methods. While the HM-Index requires additional storage for its tree structure and node-level MLPs, the overall footprint remains efficient due to intentionally lightweight MLP designs. Notably, the temporal k -core exhibits poor scalability in memory usage - its space requirements grow dramatically with graph size as it must store precomputed coreness values for all vertices across every time interval.

Exp-13: Effect of # of Fine-tuning Steps. We explore the influence of the fine-tuning step number on performance of COCO. Specifically, we test 5, 10, 15, 20, and 25 steps. Figures 19 and 20 show the results. We have the following observations: the performance of COCO improves monotonically with additional steps; the most significant quality gains occur between 10-15 steps; and marginal returns diminish beyond this threshold. This suggests that while COCO benefits from sufficient adaptation (≥ 15 steps), excessive fine-tuning yields limited additional improvements, establishing an effective efficiency-quality tradeoff.

Exp-14: Scalability. Figure 17 shows the scalability evaluation results of all methods on the FL and SO datasets. In this experiment, we randomly sampled vertices from the original graph to create graphs of varying sizes. As expected, the performance of all methods degrades as the graph size increases, since larger graphs usually contain larger communities. However, COCO maintains at least a $10\times$ speed advantage over all baselines. This demonstrates COCO's scalability for handling graphs of different scales.